

Ex.No.10 Use Ghidra to disassemble and analyze the malware code.

Project Objectives

- To provide a hands-on guide to disassembling and analyzing malware with Ghidra.
 - To enable readers to recognize common malware functionalities, such as persistence mechanisms, anti-analysis techniques, and network communications.
 - To equip users with the skills to interpret low-level assembly code and understand high-level behaviours.
-

GitHub Repository Components

A GitHub repository for this project could include:

1. README.md

- **Overview:** Brief description of Ghidra, its installation, and how it is used for malware analysis.
- **Requirements:** List of requirements, including Ghidra, a VM setup, and any sample binaries (preferably benign samples or detailed steps to safely acquire controlled malware samples).
- **Project Outline:** A breakdown of the tutorial steps, linking to various sections or files within the repository.

2. Tutorial Steps (Markdown or Jupyter Notebooks)

- **Step-by-step instructions, broken down into digestible sections, such as:**
 - **Environment Setup:** Instructions on setting up a virtualized and isolated environment.
 - **Initial Analysis:** How to load a binary into Ghidra, perform auto-analysis, and identify entry points.
 - **Function Analysis:** Methods for identifying and analyzing critical functions, decompiling code, and using Ghidra's cross-referencing features.
 - **String and Import Analysis:** How to locate and interpret strings and imports related to malicious behavior.
 - **Advanced Techniques:** Dynamic analysis, control flow graphs, custom scripting with Ghidra's Python or Java capabilities, and obfuscation detection.

3. Example Scripts (Python or Java)

- **Scripts to automate repetitive analysis tasks, such as:**
 - **Extracting and listing all string references.**

- Identifying network-related functions or common indicators of malware.
 - Labeling functions and data segments based on known patterns (e.g., file system manipulation, registry edits).
 - Script Examples: Python or Java code snippets demonstrating API usage in Ghidra (using Ghidra's Headless Analyzer, for example, for automation).
- 4. Sample Data (Benign Binary or Hex Dump)**
- Due to safety and ethical concerns, include only safe, benign binaries or a hex dump example rather than live malware. This can allow users to learn and practice without the risk of executing malicious code.
 - Sample binaries designed to exhibit typical malware-like behaviors, such as:
 - Executing system calls.
 - Writing to files.
 - Sending network data.
 - Alternatively, a dummy binary generator script that creates binaries with similar structures to malware but without harmful payloads.
- 5. Documentation and Comments**
- Include explanations of the code within each script, detailing what each function does.
 - Inline comments in disassembly views, providing readers with insights into assembly commands, control flow structures, and the decompiled code.
- 6. Report Template**
- A reporting template for documenting findings, which could be in Markdown or PDF format. The template should cover:
 - **Summary of Analysis:** Brief description of the suspected malware.
 - **Function Analysis:** Summary of the key functions, their purpose, and the findings.
 - **Behavioral Indicators:** Summary of malware behaviors, such as persistence, file system manipulation, or network communications.
 - **Recommendations:** Suggestions on further actions, such as containment, eradication, or further analysis.

Example Repository Structure

plaintext

ghidra-malware-analysis-project/

├── README.md

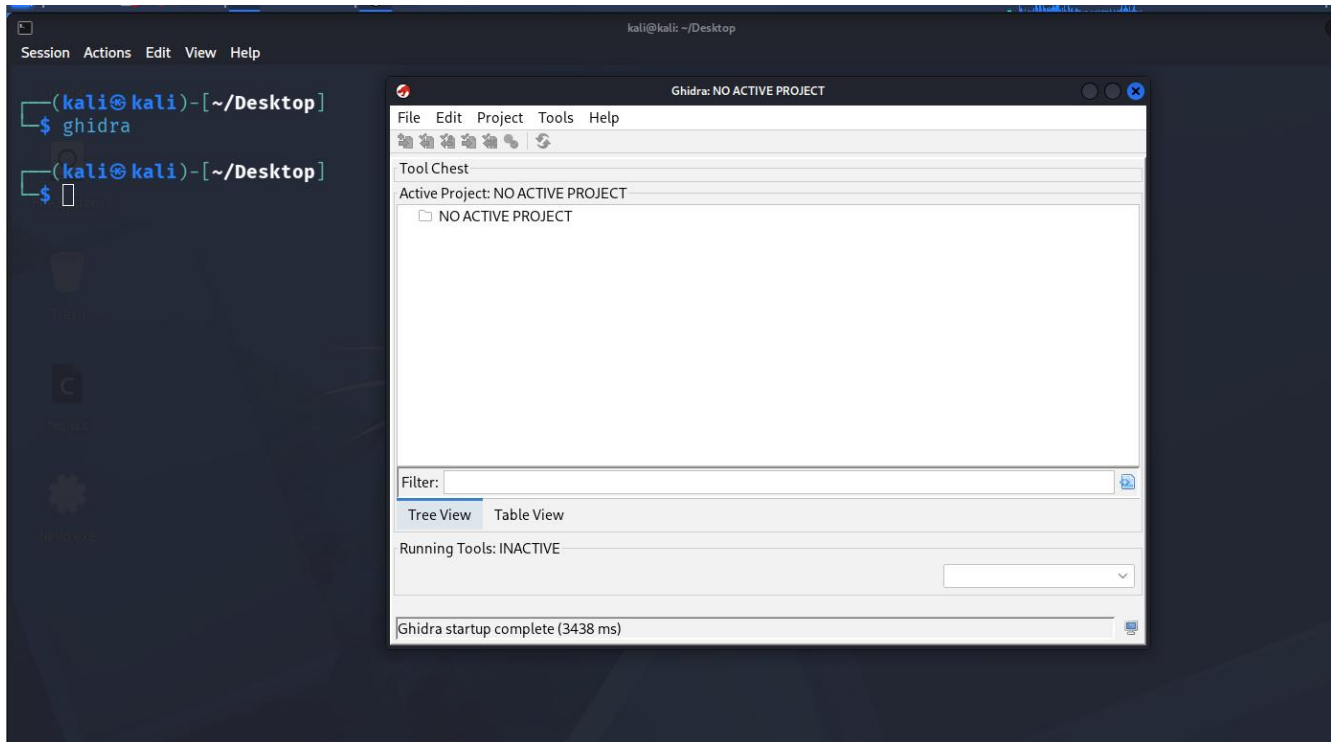
├── docs/

```
| | └─ Tutorial_Step1.md
| | └─ Tutorial_Step2.md
| | └─ Tutorial_StepN.md
| └─ scripts/
| | └─ extract_strings.py
| | └─ network_analysis.py
| | └─ label_functions.py
| └─ samples/
| | └─ benign_sample1.bin
| | └─ benign_sample2.hex
| └─ templates/
| | └─ analysis_report_template.md
└─ screenshots/
    └─ function_graph_example.png
    └─ decompiler_example.png
```

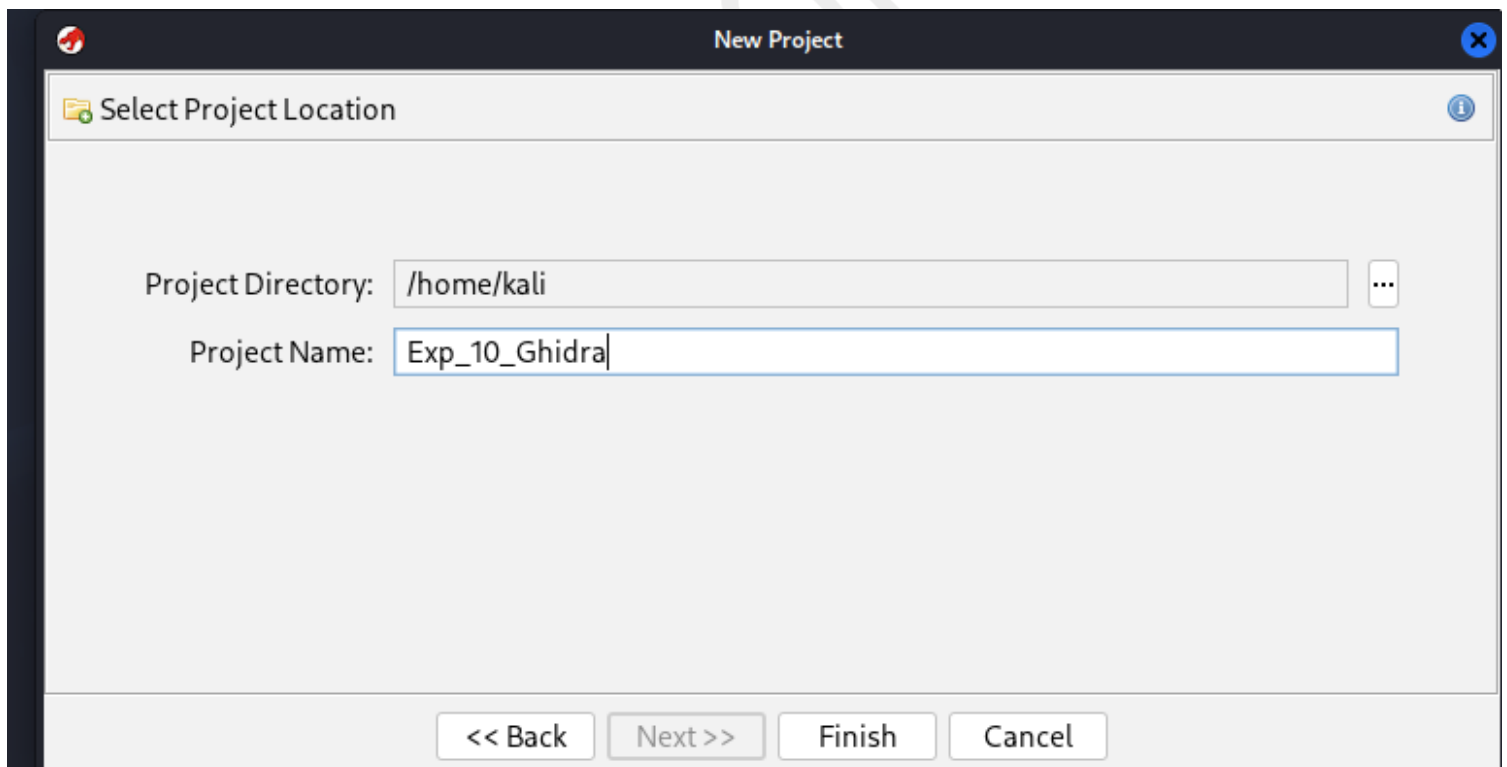
Project Development and Future Work

- **Expand Scripts:** Continue adding Ghidra scripts to automate complex analysis tasks.
- **Community Contributions:** Encourage community members to contribute additional benign samples, analysis scripts, and documentation improvements.
- **Advanced Topics:** Include guides for advanced techniques, such as unpacking obfuscated code, identifying encryption routines, and using the P-Code emulator for runtime analysis.

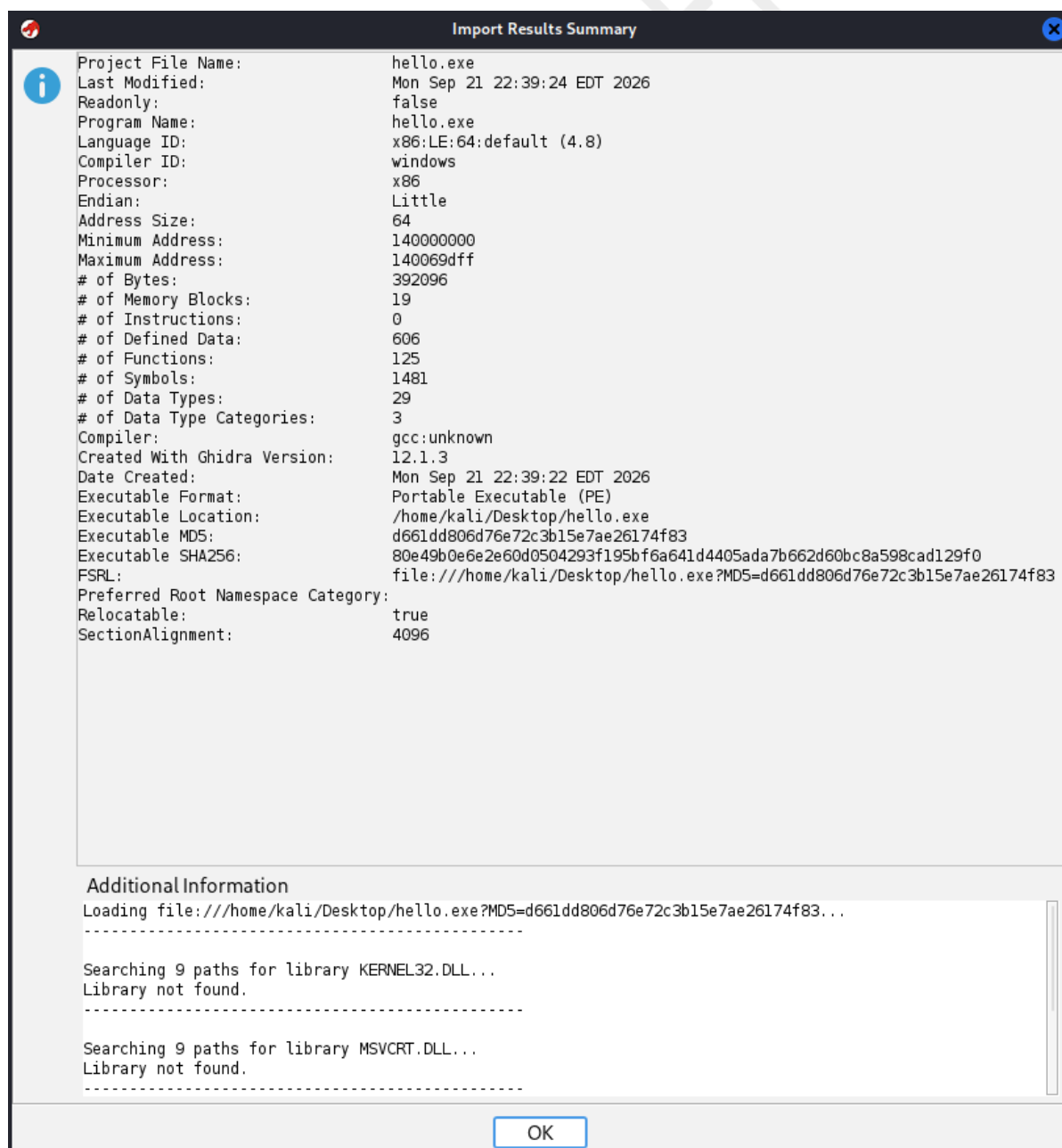
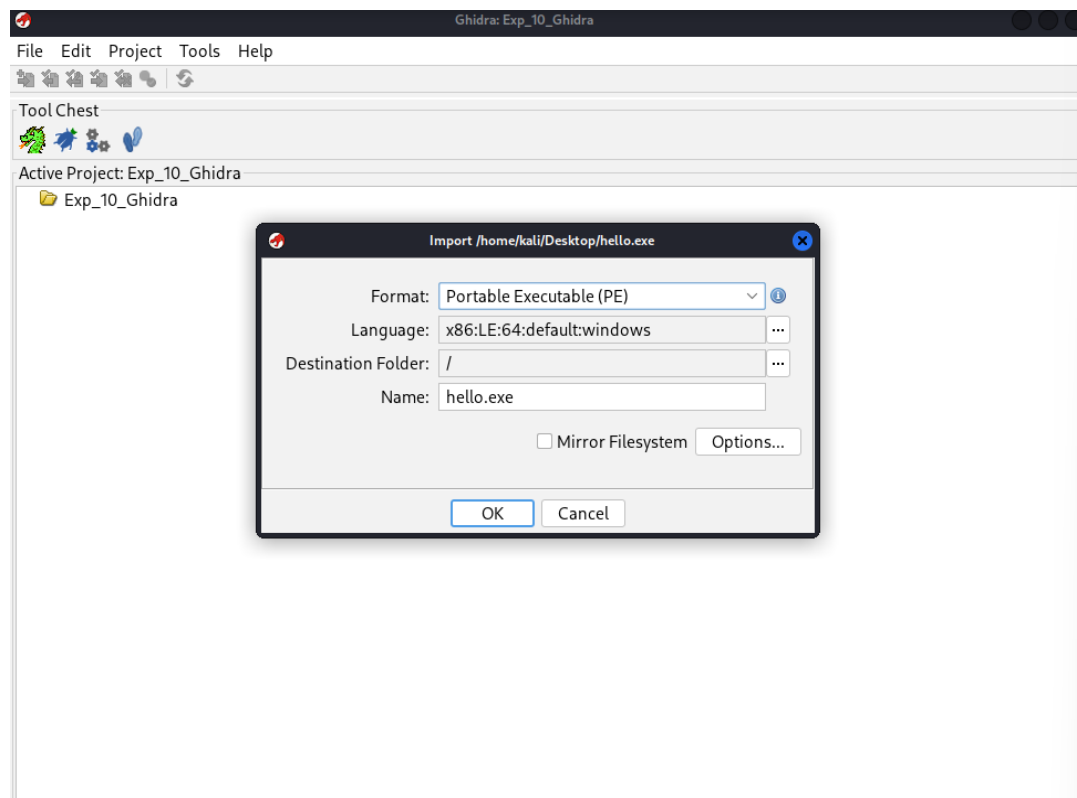
Step 1: Ghidra



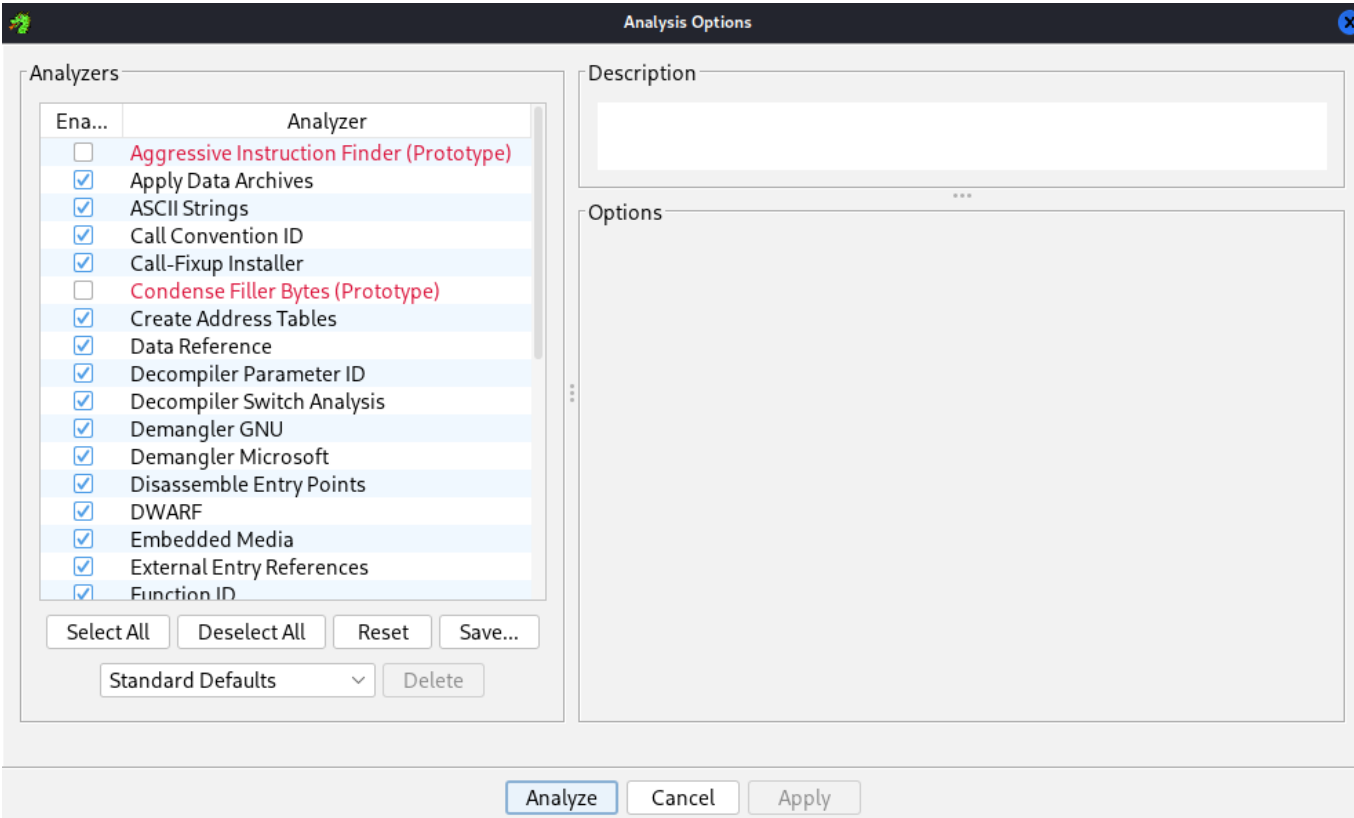
Step 2: Creating the project



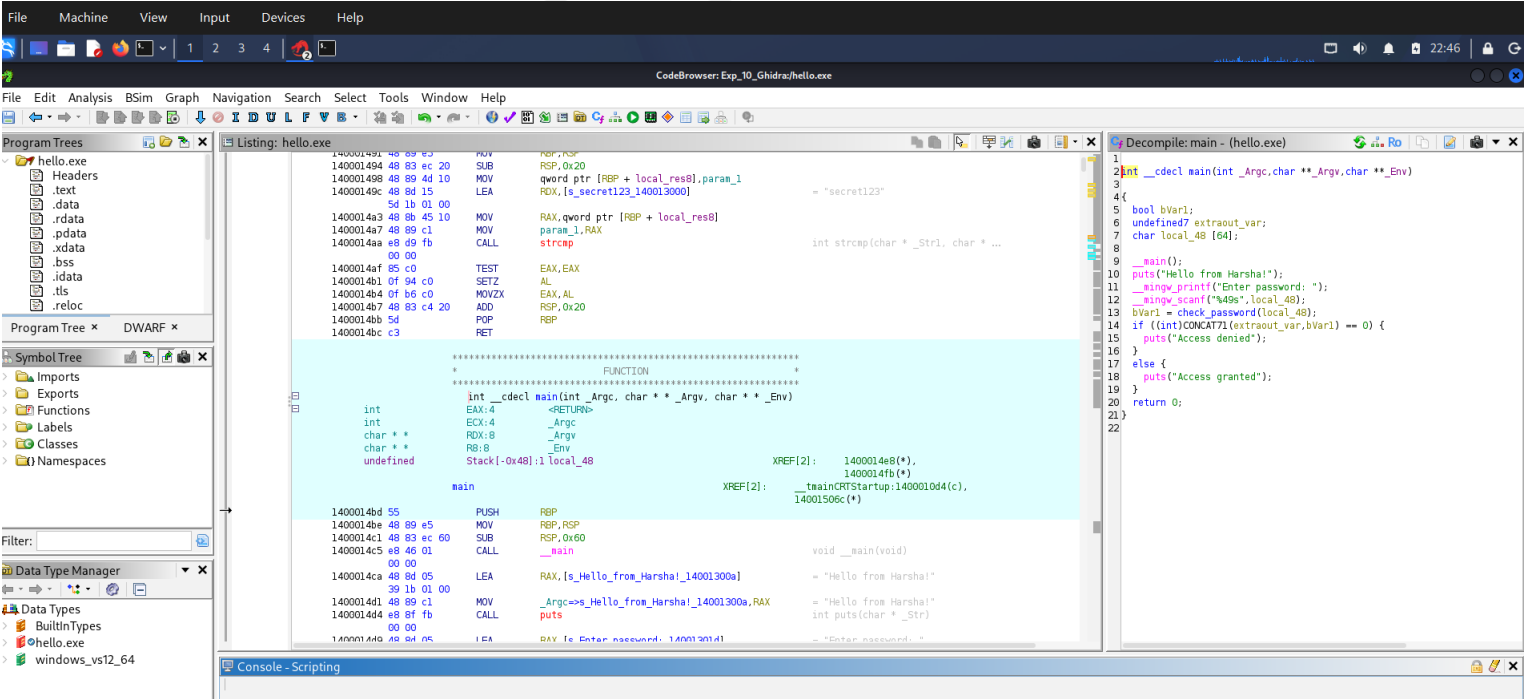
Step 3: Importing the .exe file



Step 4:Analyzing the file



Step 5:Decompiler



Step 6: Strings

Defined Strings [CodeBrowser: Exp_10_Ghidra/hello.exe]					
Defined Strings - 3013 items					
Location	String Value	String Representation	Data Type		
140000000	MZ	"MZ"	char[2]		
140000080	PE	"PE"	char[4]		
140000188	.text	".text"	char[8]		
1400001b0	.data	".data"	char[8]		
1400001d8	.rdata	".rdata"	char[8]		
140000200	.pdata	".pdata"	char[8]		
140000228	.xdata	".xdata"	char[8]		
140000250	.bss	".bss"	char[8]		
140000278	.idata	".idata"	char[8]		
1400002a0	.tls	".tls"	char[8]		
1400002c8	.reloc	".reloc"	char[8]		
1400002f0	/4	"/4"	char[8]		
140000318	/19	"/19"	char[8]		
140000340	/31	"/31"	char[8]		
140000368	/45	"/45"	char[8]		
140000390	/57	"/57"	char[8]		
1400003b8	/70	"/70"	char[8]		
1400003e0	/81	"/81"	char[8]		
140000408	/97	"/97"	char[8]		
140000430	/113	"/113"	char[8]		
140012180	secret123	"secret123"	uchar[256]		
140013000	Hello from Harsha!	"Hello from Harsha!"	ds		
14001300a	Enter password:	"Enter password: "	ds		
14001301d	Access granted	"Access granted"	ds		
140013033	Access denied	"Access denied"	ds		
140013042	Argument domain error (DOMAIN)	"Argument domain error (DOMAIN)"	ds		
1400130d0	Argument singularity (SIGN)	"Argument singularity (SIGN)"	ds		
140013100	Overflow range error (OVERFLOW)	"Overflow range error (OVERFLOW)"	ds		
140013120	Partial loss of significance (PLOSS)	"Partial loss of significance (PLOSS)"	ds		
140013148	Total loss of significance (TLOSS)	"Total loss of significance (TLOSS)"	ds		
140013170	The result is too small to be represented (UNDERFLOW)	"The result is too small to be represented (UNDERFLOW)"	ds		
1400131a6	Unknown error	"Unknown error"	ds		
1400131b8	_matherr(): %s in %s(%g, %g) (retval=%g)	"_matherr(): %s in %s(%g, %g) (retval=%g)\n"	ds		
140013200	Mingw-w64 runtime failure:	"Mingw-w64 runtime failure:\n"	ds		
140013220	Address %p has no image-section	"Address %p has no image-section"	ds		
140013240	VirtualQuery failed for %d bytes at address %p	" VirtualQuery failed for %d bytes at address %p"	ds		
140013278	VirtualProtect failed with code 0x%x	" VirtualProtect failed with code 0x%x"	ds		

Step 7: Funtions

Program Trees

hello.exe

Headers

.text

.data

.rdata

.pdata

.xdata

.bss

.idata

.tls

.reloc

Program Tree x DWARF x

Symbol Tree

Imports

Exports

Functions

Labels

Classes

Namespaces

Filter:

Data Type Manager

Data Types

BuiltInTypes

hello.exe

windows_vs12_64

Listing: hello.exe

LAB_140001050

crtxe.c:163 (9)

XREF[1]: 140001067(j)

140001050 48 39 c6 CMP RSI,RAX

140001053 0f 84 af JZ LAB_140001108

00 00 00

crtxe.c:168 (7)

140001059 b9 e8 03 MOV ECX,0x3e8

00 00 00

14000105e ff d7 CALL RDI=>KERNEL32.DLL::Sleep

LAB_140001060

crtxe.c:161 (9)

XREF[1]: 14000104b(j)

intrin-impl.h:1759 (9)

intrin-impl.h:1760 (9)

140001060 31 c0 XOR EAX,EAX

140001062 f0 48 0f CMPSQ qword ptr [RAX]>__native_startup_lock,RSI = NaP

b1 33

140001067 75 e7 JNZ LAB_140001050

crtxe.c:157 (3)

140001069 45 31 ed XOR R13D,R13D

LAB_14000106c

crtxe.c:170 (10)

XREF[1]: 14000110e(j)

14000106c 48 8b 2d MOV RBP,qword ptr [>__native_startup_state] = 140017048

0d 2d 01 00

140001073 8b 45 00 MOV EAX,dword ptr [RBP]>__native_startup_state = ??

crtxe.c:170 (9)

140001078 83 f8 01 CMP EAX,0x1

140001079 0f 84 8f JZ LAB_140001140e

03 00 00

crtxe.c:174 (3)

14000107f 8b 45 00 MOV EAX,dword ptr [RBP]>__native_startup_state = ??

crtxe.c:174 (8)

140001082 85 c0 TEST EAX,EAX

140001084 0f 84 b6 JZ LAB_140001140

00 00 00

crtxe.c:248 (10)

14000108a c7 05 70 MOV dword ptr [base.crtdll] ,0x1 = ??

00 00 00

Console - Scripting

Functions - 186 items

Name

Function...

Function Size

__mingw_invalidParamet...

1.. void ...

1

safe_flush

1.. void ...

7

__tmainCRTStartup

1.. int ...

982

WinMainCRTStartup

1.. int W...

29

mainCRTStartup

1.. int m...

29

atexit

1.. int a...

5

__gcc_register_frame

1.. undef...

12

__gcc_deregister_frame

1.. undef...

1

check_password

1.. bool ...

45

main

1.. int m...

121

__do_global_dtors

1.. void ...

66

__do_global_ctors

1.. void ...

111

__main

1.. void ...

26

__setargv

1.. int ...

3

__dyn_tls_dtor

1.. void ...

15

__dyn_tls_init

1.. void ...

110

__tregdtor

1.. int ...

3

__matherr

1.. int ...

218

__report_error

1.. void ...

93

mark_section_writable

1.. void ...

346

__pei386_runtime_relocat...

1.. void ...

886

__mingw_raise_mather...

1.. void ...

62

__mingw_setusermather...

1.. void ...

12

__mingw_SEH_error_han...

1.. int ...

357

__gnu_exception_handler

1.. long ...

395

__mingwthr_run_key_dto...

1.. void ...

106

w64_mingwthr_add

1.. int ...

134

Filter:

Decompiler x Functions x

Step 8:Exporting the results

